Android RPA 开发文档

slug: /NewMYTOS/MYT_ANDROID_RPA

Android RPA 开发文档

MYT RPA SDK Python 语言 API 文档

> 该包封装了 libmytrpc.dll 中的所有功能，用于远程控制设备

下载连接

📌 端口列表

- 更新日期 2025-04-07 初始版本

名称	版本	更新时间	操作
RPA-SDK	20251009	2025-10-09 15:58:15	下载

桥接模式

- **IP 地址**：桥接设备的 IP 地址
- **RPA 端口**：固定为 **9083**

非桥接模式

- **IP 地址**（如 192.168.30.2）：用来定位具体的设备或宿主机
- **RPA 端口**：根据设备实例索引计算得出，计算公式为：**30000 + (index - 1) × 100 + 2**

Q1 设备端口列表（index 1-12）：

实例位	实例位(index)	RPA 端口
坑位1	1	30002
坑位2	2	30102
坑位3	3	30202
坑位4	4	30302
坑位5	5	30402
坑位6	6	30502
坑位7	7	30602
坑位8	8	30702
坑位9	9	30802
坑位10	10	30902
坑位11	11	31002
坑位12	12	31102

P1 设备端口列表（index 1-24）：

实例位	坑位号(index)	RPA 端口
坑位1	1	30002
坑位2	2	30102
坑位3	3	30202
坑位4	4	30302
坑位5	5	30402

实例位	坑位号(index)	RPA 端口
坑位6	6	30502
坑位7	7	30602
坑位8	8	30702
坑位9	9	30802
坑位10	10	30902
坑位11	11	31002
坑位12	12	31102
坑位13	13	31202
坑位14	14	31302
坑位15	15	31402
坑位16	16	31502
坑位17	17	31602
坑位18	18	31702
坑位19	19	31802
坑位20	20	31902
坑位21	21	32002
坑位22	22	32102
坑位23	23	32202
坑位24	24	32302

目录结构

以下是 MYT RPA SDK 的完整目录结构，帮助您理解如何组织和使用 SDK 文件：

```
MYT_RPA_SDK_V10_20250407/
├── arm/                    # ARM架构库文件
│   └── libmytrpc_arm64.so  # ARM64 Linux库
├── centos7/                # CentOS 7库文件
│   └── libmytrpc_centos.so # CentOS 7 Linux库
├── demo_go/                # Go语言演示项目
│   ├── go.mod              # Go模块定义
│   ├── libmytrpc.dll       # Windows库文件
│   ├── libmytrpc_arm64.so  # ARM64 Linux库
│   ├── main.go             # Go演示主程序
│   ├── mytrpc_demo.exe     # 编译后的Go可执行文件
│   └── mytrpc/             # Go语言SDK封装
│       ├── API文档.md       # Go SDK API文档
│       ├── embed_linux.go   # Linux嵌入实现
│       ├── embed_windows.go # Windows嵌入实现
│       ├── libmytrpc.dll    # Windows库文件
│       ├── mytrpc_linux.go  # Linux SDK实现
│       ├── mytrpc_windows.go # Windows SDK实现
│       └── libs/            # 依赖库目录
├── demo_py_x64/            # Python 64位演示项目
│   ├── rpc_demo.py         # Python演示脚本
│   ├── common/             # 公共模块目录
│   │   ├── logger.py       # 日志模块
│   │   ├── mytRpc.py       # Python SDK核心实现
│   │   ├── mytSelector.py  # 选择器实现
│   │   ├── rpcNode.py      # 节点操作实现
│   │   ├── ToolsKit.py     # 工具函数
│   │   └── __init__.py     # 模块初始化
│   ├── lib/                # 库文件目录
│   │   ├── libmytrpc.dll   # Windows库文件
│   │   ├── libmytrpc.dylib # macOS库文件
│   │   └── libmytrpc.lib   # Windows导入库
│   └── log/                # 日志目录
│       └── myt.log         # 日志文件
├── include/                # 头文件目录
│   └── libmytrpc.h         # C头文件
├── lib/                    # Windows 32位库目录
│   ├── libmytrpc.dll       # Windows 32位库文件
│   └── libmytrpc.lib       # Windows 32位导入库
├── lib64/                  # Windows 64位库目录
│   ├── libmytrpc.dll       # Windows 64位库文件
│   └── libmytrpc.lib       # Windows 64位导入库
```

```
├── macos(m)/                # macOS M系列芯片库目录
│   └── libmytrpc.dylib      # macOS M系列库文件
├── macos(x86)/              # macOS x86芯片库目录
│   └── libmytrpc.dylib      # macOS x86库文件
├── ubuntu/                  # Ubuntu库目录
│   └── libmytrpc_ubuntu_x86_64.so  # Ubuntu x86_64库文件
├── demo_go.zip              # Go演示项目压缩包
└── README.txt               # SDK说明文档
```

引用文件功能与使用说明

模块导入

根据实际的 SDK 实现，Python 版本采用面向对象的方式进行调用。以下是正确的导入方式：

```python
# 导入核心SDK类
from common.mytRpc import MytRpc
# 导入选择器类
from common.mytSelector import mytSelector
# 导入节点操作类
from common.rpcNode import rpcNode
```

依赖要求

- Python 3.7+ 64位版本
- Windows 操作系统（当前SDK主要支持Windows）
- 需要将 `libmytrpc.dll` 文件放置在项目的 `common` 目录下或系统 PATH 目录
- 可选依赖：
  - OpenCV (cv2)：用于图像处理和显示
  - NumPy (numpy)：用于图像数据处理

引用文件功能说明

以下是 SDK 中各个引用文件的详细功能介绍：

文件名	功能描述	主要方法/类	使用场景
`mytRpc.py`	核心 SDK 实现，封装了与设备通信的所有主要功能	`MytRpc` 类	所有设备控制操作的入口点
`mytSelector.py`	提供 UI 元素选择器功能，用于查找和筛选 UI 节点	`mytSelector` 类	需要根据条件查找 UI 元素时使用
`rpcNode.py`	封装了 UI 节点的操作和属性获取方法	`rpcNode` 类	获取节点属性、执行节点操作时使用
`logger.py`	提供日志记录功能	`logger` 对象	需要记录日志信息时使用
`ToolsKit.py`	提供各种工具函数	`ToolsKit` 类	获取程序路径、检查进程状态等工具操作
`__init__.py`	模块初始化文件	-	确保 common 目录可作为 Python 模块导入
`libmytrpc.dll`	底层动态链接库，实现了与设备通信的核心功能	-	提供 SDK 核心功能的底层实现，所有上层操作最终都会调用此 DLL 中的函数

DLL 文件详细说明

libmytrpc.dll - 核心动态链接库

功能描述：`libmytrpc.dll` 是 MYT RPA SDK 的核心动态链接库，实现了与远程设备通信的所有底层功能，包括设备连接、截图、触摸操作、按键输入、UI 节点获取等。

文件位置：

- 在 SDK 包中的位置：`MYT_RPA_SDK_V10_20250407/demo_py_x64/lib/libmytrpc.dll`
- 在项目中的位置：项目的 `common` 目录下，即 `项目目录/common/libmytrpc.dll`

引用方式：

1. **自动引用**：SDK 会自动在以下位置查找 `libmytrpc.dll`：
    - 当前 Python 脚本所在目录的 `common` 子目录
    - 系统 PATH 环境变量指定的目录
2. **手动设置路径**：如果需要使用自定义路径的 DLL 文件，可以在创建 `MytRpc` 实例前修改代码中的 DLL 路径设置。

使用说明：

- 开发者无需直接调用 DLL 中的函数，而是通过 `MytRpc` 类提供的方法间接使用 DLL 功能
- DLL 文件会在创建 `MytRpc` 实例时自动加载
- 不同平台有对应的库文件：
    - Windows: `libmytrpc.dll`
    - Linux: `libmytrpc.so` （不同架构有不同版本）
    - macOS: `libmytrpc.dylib`

DLL 依赖关系：

- Windows 平台：需要 Visual C++ Redistributable 运行时库
- Linux 平台：需要相应的系统依赖库
- macOS 平台：需要相应的系统框架支持

使用示例：

```python
from common.mytRpc import MytRpc

# 创建 MytRpc 实例时，会自动加载 libmytrpc.dll
mytapi = MytRpc()

# 以下操作都会间接调用 DLL 中的功能
if mytapi.init("192.168.1.100", 30202, 10) == True:
    # 调用 DLL 中的截图功能
    mytapi.screentshot(1, 90, "screenshot.png")

    # 调用 DLL 中的触摸操作功能
    mytapi.touchClick(0, 500, 500)
```

常见问题与解决方案：

1. **找不到 DLL 文件**：

    - 确保 DLL 文件已正确复制到项目的 `common` 目录
    - 检查系统 PATH 环境变量是否包含 DLL 所在目录
    - 检查 DLL 文件是否与当前 Python 版本（32位/64位）匹配

2. **DLL 加载失败**：
    - 检查是否缺少依赖库（如 Visual C++ Redistributable）
    - 确保 DLL 文件未被损坏
    - 检查当前用户是否有足够的权限访问 DLL 文件

3. **DLL 版本不匹配**：
    - 确保使用的 DLL 文件与 SDK 版本一致
    - 不要混合使用不同版本的 DLL 文件

DLL 功能模块：

1. **设备通信模块**：负责与远程设备建立和维护连接
2. **截图模块**：实现远程设备截图功能
3. **输入控制模块**：实现触摸、按键等输入操作
4. **UI 树模块**：获取和解析设备 UI 树结构
5. **节点操作模块**：实现 UI 节点的查找和操作
6. **App 管理模块**：实现应用的打开、关闭等操作
7. **视频流模块**：实现设备屏幕视频流传输

性能优化建议：

- 避免频繁创建和销毁 `MytRpc` 实例，这会导致 DLL 频繁加载和卸载
- 对于频繁执行的操作，考虑批量处理
- 合理设置超时时间，避免不必要的等待
- 及时释放不再使用的资源，如选择器和节点对象

各文件详细介绍

1. mytRpc.py - 核心 SDK 实现

`MytRpc` 类是 SDK 的核心类，负责与远程设备建立连接并提供所有设备控制功能。

主要功能：

- 设备连接与管理
- 截图操作
- 触摸操作
- 按键输入
- App 操作
- 节点树导出
- 选择器创建与管理

使用示例:

```
from common.mytRpc import MytRpc

# 创建 SDK 实例
mytapi = MytRpc()

# 初始化并连接设备
if mytapi.init("192.168.1.100", 30202, 10) == True:
    print("设备连接成功")

    # 执行设备命令
    output, result = mytapi.exec_cmd("ls -la")
    print(f"命令输出: {output}")

    # 截图保存
    mytapi.screentshot(1, 90, "screenshot.png")
```

2. mytSelector.py - UI 元素选择器

`mytSelector` 类用于创建和管理 UI 元素选择条件，查找符合条件的 UI 节点。

主要功能:

- 支持多种条件筛选（ID、文本、类名、包名等）
- 支持布尔属性筛选（可点击、可见、可滚动等）
- 支持边界条件筛选
- 支持正则表达式匹配

使用示例:

```
from common.mytRpc import MytRpc
```

```
mytapi = MytRpc()
mytapi.init("192.168.1.100", 30202, 10)

# 创建选择器
selector = mytapi.create_selector()
with selector:
    # 设置选择条件
    selector.addQuery_TextContainWith('登录')  # 文本包含"登录"
    selector.addQuery_Clickable(True)  # 可点击

    # 查找节点
    node = selector.execQueryOne(2000)  # 超时2000毫秒
    if node is not None:
        print("找到匹配节点")
        node.Click_events()  # 点击节点
```

3. rpcNode.py - UI 节点操作

`rpcNode` 类封装了 UI 节点的属性获取和操作方法。

主要功能:

- 获取节点属性（文本、描述、ID、类名等）
- 获取节点位置和边界信息
- 节点点击和长按操作
- 获取节点父子关系

使用示例:

```
# 假设已通过选择器获取到节点 node

# 获取节点属性
node_json = node.getNodeJson()
node_text = node.getNodeText()
node_id = node.getNodeId()

# 获取节点位置
bounds = node.getNodeNound()
print(f"节点位置: {bounds}")

# 点击节点
node.Click_events()
```

```
# 长按节点
node.longClick_events()
```

4. logger.py - 日志记录

提供日志记录功能，支持控制台和文件输出。

主要功能：

- 支持多种日志级别（debug、info、warning、error、crit）
- 自动按天分割日志文件
- 同时输出到控制台和文件

使用示例：

```
from common.logger import logger

logger.debug("调试信息")
logger.info("普通信息")
logger.warning("警告信息")
logger.error("错误信息")
```

5. ToolsKit.py - 工具函数

提供各种辅助工具函数。

主要功能：

- 获取程序根路径
- 检查进程是否存在
- 防止程序多次运行

使用示例：

```
from common.ToolsKit import ToolsKit

tools = ToolsKit()

# 获取程序根路径
root_path = tools.GetRootPath()
print(f"程序根路径: {root_path}")
```

```
# 检查进程是否存在
is_running = tools.check_process(1234)
print(f"进程1234是否存在: {is_running}")
```

安装说明

1. **复制库文件**：从 `demo_py_x64/lib/` 目录复制对应平台的库文件到您项目的 `common` 目录：

```
项目目录/
└── common/
    └── libmytrpc.dll  # Windows库文件
```

2. **复制Python模块**：从 `demo_py_x64/common/` 目录复制以下 Python 模块到您项目的 `common` 目录：
   - `mytRpc.py`（核心SDK实现）
   - `mytSelector.py`（选择器功能）
   - `rpcNode.py`（节点操作）
   - `logger.py`（日志功能）
   - `ToolsKit.py`（工具函数）
   - `__init__.py`（模块初始化）
3. **项目结构**：最终的项目结构应如下所示：

```
项目目录/
├── common/
│   ├── __init__.py
│   ├── logger.py
│   ├── libmytrpc.dll
│   ├── mytRpc.py
│   ├── mytSelector.py
│   ├── rpcNode.py
│   └── ToolsKit.py
└── your_script.py  # 您的Python脚本
```

4. **在脚本中使用**：在您的Python脚本中按如下方式使用SDK：

```
from common.mytRpc import MytRpc
```

```
# 创建SDK实例
mytapi = MytRpc()

# 初始化并连接设备
if mytapi.init("设备IP", 端口号, 超时时间) == True:
    print("连接成功")
```

目录

- 初始化与释放
- 基础设备方法
- 截图方法
- 触摸操作方法
- 按键和文本输入方法
- App操作方法
- 视频流方法
- 选择器方法
- 节点集合操作方法
- 节点属性获取方法
- 节点操作方法
- 选择器筛选方法
  - 布尔属性筛选
  - 边界筛选
  - ID匹配筛选
  - Text匹配筛选
  - Class匹配筛选
  - Package匹配筛选
  - Desc匹配筛选

数据结构

CaptureResult - 截图结果

字段	类型	说明
data	bytes	图像数据（RGBA格式）
width	int	图像宽度（像素）
height	int	图像高度（像素）
stride	int	图像步长（每行字节数）
ptr	int	原始指针，需要调用 free_rpc_ptr 释放

CompressedCaptureResult - 压缩截图结果

字段	类型	说明
data	bytes	压缩后的图像数据（PNG或JPG格式）
ptr	int	原始指针，需要调用 free_rpc_ptr 释放

Bounds - 节点边界

字段	类型	说明
left	int	左边界坐标
top	int	上边界坐标
right	int	右边界坐标
bottom	int	下边界坐标

Point - 坐标点

字段	类型	说明
x	int	X坐标
y	int	Y坐标

初始化与释放

init

初始化 DLL，加载所有函数

```
def init(dll_path: str = "") -> None:
```

参数	类型	说明
dll_path	str	DLL文件的路径，如果为空则使用默认路径 "libmytrpc.dll"

异常: Exception - 加载失败时抛出异常

release

释放 DLL

```
def release() -> None:
```

异常: Exception - 释放失败时抛出异常

基础设备方法

get_sdk_version

获取当前库的版本号

```
def get_sdk_version() -> int:
```

返回值: int - 版本号

openDevice

远程连接设备

```
def openDevice(ip: str, port: int, timeout: int) -> int:
```

参数	类型	说明
ip	str	要远程控制的设备的IP地址
port	int	要远程控制的设备的端口
timeout	int	远程连接的超时时间，单位秒

返回值: int - 句柄id，大于0表示成功，失败返回0

closeDevice

关闭远程连接

```
def closeDevice(handle: int) -> int:
```

参数	类型	说明
handle	int	openDevice返回的id

返回值: int - 0表示成功，失败返回非0

check_connect_state

检测远程连接是否处于连接状态

```
def check_connect_state(handle: int) -> bool:
```

参数	类型	说明
handle	int	openDevice返回的id

返回值: int - 1表示已连接，0表示已断开

free_rpc_ptr

释放截图数据

```
def free_rpc_ptr(ptr: int) -> None:
```

参数	类型	说明
ptr	int	指针数据

get_display_rotate

获取当前屏幕的旋转角度

```
def get_display_rotate(handle: int) -> int:
```

参数	类型	说明
handle	int	openDevice返回的id

返回值: int - 返回0,1,2,3

exec_cmd

执行shell命令

```
def exec_cmd(handle: int, wait_for_exit: bool, cmdline: str) -> str:
```

参数	类型	说明
handle	int	openDevice返回的id
wait_for_exit	bool	是否等待执行结束才返回
cmdline	str	命令行

返回值: str - 命令执行结果字符串

dumpNodeXml

获取节点树数据（XML格式）

```
def dumpNodeXml(handle: int, dump_all: bool) -> str:
```

参数	类型	说明
handle	int	openDevice返回的id
dump_all	bool	是否导出所有节点

返回值: str - 节点树XML字符串

dump_node_xml_ex

获取节点树数据（扩展版本）

```
def dump_node_xml_ex(handle: int, use_new_mode: bool, timeout: int) -> str:
```

参数	类型	说明
handle	int	openDevice返回的id
use_new_mode	bool	是否使用新模式
timeout	int	超时时间

返回值: str - 节点树XML字符串

use_new_node_mode

设置节点模式

```
def use_new_node_mode(handle: int, use: bool) -> int:
```

参数	类型	说明
handle	int	openDevice返回的id
use	bool	是否使用新模式

返回值: int - 操作结果

截图方法

take_capture

远程截图，获取到的是RGBA的数据流

```
def take_capture(handle: int) -> CaptureResult:
```

参数	类型	说明
handle	int	openDevice返回的id

返回值: CaptureResult - 截图结果

异常: Exception - 截图失败时抛出异常

take_capture_ex

远程截图（指定区域）

```
def take_capture_ex(handle: int, l: int, t: int, r: int, b: int) ->
CaptureResult:
```

参数	类型	说明
handle	int	openDevice返回的id
l	int	截图区域的左坐标
t	int	截图区域的上坐标
r	int	截图区域的右坐标
b	int	截图区域的下坐标

返回值: CaptureResult - 截图结果

异常: Exception - 截图失败时抛出异常

takeCaptrueCompress

远程截图，获取压缩后的png或jpg格式

```
def takeCaptrueCompress(handle: int, image_type: int, quality: int) ->
CompressedCaptureResult:
```

参数	类型	说明
handle	int	openDevice返回的id
image_type	int	0表示png，1表示jpg
quality	int	压缩质量，取值0-100

返回值: CompressedCaptureResult - 压缩截图结果

异常: Exception - 截图失败时抛出异常

setRpaWorkMode

设置RPA工作模式（无障碍模式开关）

```
def setRpaWorkMode(mode: int) -> bool:
```

参数	类型	说明
mode	int	工作模式（0: 关闭无障碍, 1: 开启无障碍）

返回值: bool - 设置成功返回True，失败返回False

说明:

- 该方法用于设置RPA的工作模式，主要是控制是否使用无障碍模式
- 开启无障碍模式后，可以获取更加完整的节点信息，但某些应用环境会检测是否开启了无障碍
- 该方法需要最新的固件版本支持

take_capture_compress_ex

远程截图（指定区域，压缩格式）

```
def take_capture_compress_ex(handle: int, l: int, t: int, r: int, b: int,
image_type: int, quality: int) -> CompressedCaptureResult:
```

参数	类型	说明
handle	int	openDevice返回的id
l	int	截图区域的左坐标
t	int	截图区域的上坐标
r	int	截图区域的右坐标
b	int	截图区域的下坐标
image_type	int	0表示png，1表示jpg
quality	int	压缩质量，取值0-100

返回值: CompressedCaptureResult - 压缩截图结果

异常: Exception - 截图失败时抛出异常

触摸操作方法

touchDown

模拟按下

```
def touchDown(handle: int, id: int, x: int, y: int) -> int:
```

参数	类型	说明
handle	int	openDevice返回的id
id	int	手指编号(0-9)
x	int	X坐标
y	int	Y坐标

返回值: int - 操作结果

touchUp

模拟弹起

```
def touchUp(handle: int, id: int, x: int, y: int) -> int:
```

参数	类型	说明
handle	int	openDevice返回的id
id	int	手指编号(0-9)
x	int	X坐标
y	int	Y坐标

返回值: int - 操作结果

touchMove

模拟滑动

```
def touchMove(handle: int, id: int, x: int, y: int) -> int:
```

参数	类型	说明
handle	int	openDevice返回的id
id	int	手指编号(0-9)
x	int	X坐标
y	int	Y坐标

返回值: int - 操作结果

touchClick

模拟单击

```
def touchClick(handle: int, id: int, x: int, y: int) -> int:
```

参数	类型	说明
handle	int	openDevice返回的id
id	int	手指编号(0-9)
x	int	X坐标
y	int	Y坐标

返回值: int - 操作结果

swipe

模拟滑动

```
def swipe(handle: int, id: int, x0: int, y0: int, x1: int, y1: int, millis:
```

```
int, async: bool) -> None:
```

参数	类型	说明
handle	int	openDevice返回的id
id	int	手指编号(0-9)
x0	int	起始X坐标
y0	int	起始Y坐标
x1	int	结束X坐标
y1	int	结束Y坐标
millis	int	滑动持续时间（毫秒）
async	bool	是否异步执行

按键和文本输入方法

keyPress

模拟按键

```
def keyPress(handle: int, code: int) -> int:
```

参数	类型	说明
handle	int	openDevice返回的id
code	int	按键码

返回值: int - 操作结果

sendText

模拟键盘输入

```
def sendText(handle: int, text: str) -> int:
```

参数	类型	说明
handle	int	openDevice返回的id
text	str	要输入的字符串

返回值: int - 操作结果

App操作方法

openApp

打开指定包名的app

```
def openApp(handle: int, pkg: str) -> int:
```

参数	类型	说明
handle	int	openDevice返回的id
pkg	str	包名

返回值: int - 操作结果

stopApp

停止指定的应用

```
def stopApp(handle: int, pkg: str) -> int:
```

参数	类型	说明
handle	int	openDevice返回的id
pkg	str	包名

返回值: int - 操作结果

视频流方法

start_video_stream

启动屏幕视频流传输

```
def start_video_stream(handle: int, w: int, h: int, bitrate: int) -> int:
```

参数	类型	说明
handle	int	openDevice返回的id
w	int	希望输出的宽度
h	int	希望输出的高度
bitrate	int	输出的比特率

返回值: int - 操作结果

> 注意: 视频流回调函数的实现需要通过ctypes来完成

stop_video_stream

关闭屏幕视频流

```
def stop_video_stream(handle: int) -> int:
```

参数	类型	说明
handle	int	openDevice返回的id

返回值: int - 操作结果

选择器方法

create_selector

创建一个筛选器

```
def create_selector(handle: int) -> int:
```

参数	类型	说明
handle	int	openDevice返回的id

返回值: int - 筛选器唯一标识号

clear_selector

清空筛选器中所有的筛选条件

```
def clear_selector(sel: int) -> None:
```

参数	类型	说明
sel	int	new_selector返回的筛选器编号

free_selector

释放筛选器

```
def free_selector(sel: int) -> None:
```

参数	类型	说明
sel	int	new_selector返回的筛选器编号

find_nodes

使用筛选器去查找

```
def find_nodes(sel: int, max_cnt_ret: int, timeout: int) -> int:
```

参数	类型	说明
sel	int	new_selector返回的筛选器编号
max_cnt_ret	int	最大返回节点数
timeout	int	查找超时时间（毫秒）

返回值: int - 结果集唯一标识编号

free_nodes

释放结果集

```
def free_nodes(nodes: int) -> None:
```

参数	类型	说明
nodes	int	find_nodes返回的结果集

节点集合操作方法

get_nodes_size

获取结果集中节点的个数

```
def get_nodes_size(nodes: int) -> int:
```

参数	类型	说明
nodes	int	find_nodes返回的结果集

返回值: int - 节点个数

get_node_by_index

按照顺序从结果集中获取节点

```
def get_node_by_index(nodes: int, index: int) -> int:
```

参数	类型	说明
nodes	int	find_nodes返回的结果集
index	int	节点索引

返回值: int - 节点句柄

get_node_parent

获取给定节点的父节点句柄

```
def get_node_parent(node: int) -> int:
```

参数	类型	说明
node	int	节点句柄

返回值: int - 父节点句柄

get_node_child_count

获取给定节点的子节点数量

```
def get_node_child_count(node: int) -> int:
```

参数	类型	说明
node	int	节点句柄

返回值: int - 子节点数量

get_node_child

获取给定节点的子节点

```
def get_node_child(node: int, index: int) -> int:
```

参数	类型	说明
node	int	节点句柄
index	int	子节点索引

返回值: int - 子节点句柄

节点属性获取方法

get_node_json

获取节点的JSON字符串格式

```
def get_node_json(node: int) -> str:
```

参数	类型	说明
node	int	节点句柄

返回值: str - JSON字符串

get_node_text

获取节点的文本属性

```
def get_node_text(node: int) -> str:
```

参数	类型	说明
node	int	节点句柄

返回值: str - 文本内容

get_node_desc

获取节点的描述属性

```
def get_node_desc(node: int) -> str:
```

参数	类型	说明
node	int	节点句柄

返回值: str - 描述内容

get_node_package

获取节点的包名属性

```
def get_node_package(node: int) -> str:
```

参数	类型	说明
node	int	节点句柄

返回值: str - 包名

get_node_class

获取节点的类名属性

```
def get_node_class(node: int) -> str:
```

参数	类型	说明
node	int	节点句柄

返回值: str - 类名

get_node_id

获取节点的资源ID属性

```
def get_node_id(node: int) -> str:
```

参数	类型	说明
node	int	节点句柄

返回值: str - 资源ID

get_node_bound

获取节点的范围属性

```
def get_node_bound(node: int) -> Bounds:
```

参数	类型	说明
node	int	节点句柄

返回值: Bounds - 边界信息

异常: Exception - 获取失败时抛出异常

get_node_bound_center

获取节点的中心坐标

```
def get_node_bound_center(node: int) -> Point:
```

参数	类型	说明
node	int	节点句柄

返回值: Point - 中心坐标

异常: Exception - 获取失败时抛出异常

节点操作方法

Click_events

点击节点

```
def Click_events(node: int) -> int:
```

参数	类型	说明
node	int	节点句柄

返回值: int - 1表示成功，0表示失败

longClick_events

长按节点

```
def longClick_events(node: int) -> int:
```

参数	类型	说明
node	int	节点句柄

返回值: int - 1表示成功，0表示失败

选择器筛选方法

布尔属性筛选

方法名	说明
`enable(sel: int, v: bool)`	设置节点是否可用筛选器
`checkable(sel: int, v: bool)`	设置节点是否可以被选中筛选器
`clickable(sel: int, v: bool)`	设置节点是否可以被点击筛选器
`focusable(sel: int, v: bool)`	设置节点是否可以获取焦点筛选器
`focused(sel: int, v: bool)`	设置节点是否已经获取焦点筛选器
`scrollable(sel: int, v: bool)`	设置节点是否可以滚动筛选器
`long_clickable(sel: int, v: bool)`	设置节点是否可以长按筛选器
`password(sel: int, v: bool)`	设置节点是否是密码筛选器
`selected(sel: int, v: bool)`	设置节点是否被选中筛选器
`visible(sel: int, v: bool)`	设置节点是否可见筛选器
`index(sel: int, v: int)`	设置节点索引筛选器

边界筛选

方法名	说明
`bounds_inside(sel: int, l: int, t: int, r: int, b: int)`	设置在节点指定范围内的筛选器
`bounds_equal(sel: int, l: int, t: int, r: int, b: int)`	设置节点的范围等于指定范围的筛选器

ID匹配筛选

方法名	说明
`id_equal(sel: int, str: str)`	设置节点的资源ID等于指定id的筛选器
`id_start_with(sel: int, str: str)`	设置节点的资源ID以指定字符串开头的筛选器
`id_end_with(sel: int, str: str)`	设置节点的资源ID以指定字符串结尾的筛选器
`id_contain_with(sel: int, str: str)`	设置节点的资源ID包含指定字符串的筛选器
`id_match_with(sel: int, str: str)`	设置节点的资源ID正则匹配指定字符串的筛选器

Text匹配筛选

方法名	说明
`text_start_with(sel: int, str: str)`	设置节点文本以指定字符串开头的筛选器
`text_end_with(sel: int, str: str)`	设置节点文本以指定字符串结尾的筛选器
`text_contain_with(sel: int, str: str)`	设置节点文本包含指定字符串的筛选器
`text_match_with(sel: int, str: str)`	设置节点文本正则匹配指定字符串的筛选器

Class匹配筛选

方法名	说明
`clz_start_with(sel: int, str: str)`	设置节点类名以指定字符串开头的筛选器
`clz_end_with(sel: int, str: str)`	设置节点类名以指定字符串结尾的筛选器
`clz_contain_with(sel: int, str: str)`	设置节点类名包含指定字符串的筛选器
`clz_match_with(sel: int, str: str)`	设置节点类名正则匹配指定字符串的筛选器

Package匹配筛选

方法名	说明
`package_start_with(sel: int, str: str)`	设置节点包名以指定字符串开头的筛选器
`package_end_with(sel: int, str: str)`	设置节点包名以指定字符串结尾的筛选器
`package_contain_with(sel: int, str: str)`	设置节点包名包含指定字符串的筛选器
`package_match_with(sel: int, str: str)`	设置节点包名正则匹配指定字符串的筛选器

Desc匹配筛选

方法名	说明
`desc_start_with(sel: int, str: str)`	设置节点描述以指定字符串开头的筛选器
`desc_end_with(sel: int, str: str)`	设置节点描述以指定字符串结尾的筛选器
`desc_contain_with(sel: int, str: str)`	设置节点描述包含指定字符串的筛选器

方法名	说明
`desc_match_with(sel: int, str: str)`	设置节点描述正则匹配指定字符串的筛选器

使用示例

基础使用示例

```
from common.mytRpc import MytRpc

def main():
    # 创建SDK实例
    mytapi = MytRpc()

    # 获取SDK版本
    sdk_ver = mytapi.get_sdk_version()
    print(f"SDK版本: {sdk_ver}")

    # 初始化并连接设备
    # 注意：端口不是adb端口，计算公式为：30000 + (index - 1) × 100 + 2
    # 例如：index=3 对应端口 30202
    if mytapi.init("192.168.1.100", 30202, 10) == True:
        print("设备连接成功")

        try:
            # 检查连接状态
            if mytapi.check_connect_state() == True:
                print("当前连接状态正常")
            else:
                print("当前连接断开")

            # 设置工作模式
            if mytapi.setRpaWorkMode(0):
                print("设置工作模式为：关闭无障碍")

        finally:
            # SDK会自动管理资源，不需要手动释放
            pass
    else:
        print("设备连接失败")
```

```
if __name__ == "__main__":
    main()
```

截图示例

```
from common.mytRpc import MytRpc
import cv2
import numpy as np

def main():
    mytapi = MytRpc()

    if mytapi.init("192.168.1.100", 30202, 10) == True:
        try:
            # 截图并保存到文件
            mytapi.screentshot(1, 90, "screenshot.png")
            print("截图保存成功")

            # 截图并进行图像处理
            byt_arr = mytapi.takeCaptrueCompress(0, 100)  # 0=PNG格式，100=质
量
            if len(byt_arr) > 0:
                # 使用NumPy和OpenCV处理图像
                img_np = np.frombuffer(byt_arr, dtype=np.uint8)
                img = cv2.imdecode(img_np, cv2.IMREAD_COLOR)

                # 显示图像
                cv2.imshow("Screenshot", img)
                cv2.waitKey(3000)  # 显示3秒
                cv2.destroyAllWindows()
                print("截图显示成功")
            else:
                print("获取截图失败")

        finally:
            pass

if __name__ == "__main__":
    main()
```

触摸操作示例

```python
from common.mytRpc import MytRpc
import time

def main():
    mytapi = MytRpc()

    if mytapi.init("192.168.1.100", 30202, 10) == True:
        try:
            # 单击操作
            print("执行单击操作")
            mytapi.click(0, 500, 500)  # 手指ID, X, Y
            time.sleep(1)

            # 长按操作
            print("执行长按操作")
            mytapi.longClick(0, 500, 500, 1.0)  # 手指ID, X, Y, 长按时间(秒)
            time.sleep(1)

            # 滑动操作
            print("执行滑动操作")
            # 从(100, 500)滑动到(500, 500)，耗时1秒
            mytapi.swipe(0, 100, 500, 500, 500, 1000)  # 手指ID, 起始X, 起始Y, 结束X, 结束Y, 持续时间(毫秒)

        finally:
            pass

if __name__ == "__main__":
    main()
```

按键和文本输入示例

```python
from common.mytRpc import MytRpc

def main():
    mytapi = MytRpc()

    if mytapi.init("192.168.1.100", 30202, 10) == True:
        try:
            # 发送文本
            print("发送文本: Hello MYT RPA!")
            if mytapi.sendText("Hello MYT RPA!"):
                print("文本发送成功")
```

```
            # 模拟按键（返回键）
            print("执行返回键操作")
            if mytapi.keyPress(4):  # 4=返回键
                print("返回键执行成功")

            # 模拟Home键
            print("执行Home键操作")
            if mytapi.keyPress(3):  # 3=Home键
                print("Home键执行成功")

        finally:
            pass

if __name__ == "__main__":
    main()
```

App操作示例

```
from common.mytRpc import MytRpc

def main():
    mytapi = MytRpc()

    if mytapi.init("192.168.1.100", 30202, 10) == True:
        try:
            # 打开应用
            app_package = "com.blue.filemanager"
            print(f"打开应用: {app_package}")
            if mytapi.openApp(app_package):
                print("应用打开成功")

            # 关闭应用
            print(f"关闭应用: {app_package}")
            if mytapi.stopApp(app_package):
                print("应用关闭成功")

        finally:
            pass

if __name__ == "__main__":
    main()
```

节点操作示例

```python
from common.mytRpc import MytRpc

def main():
    mytapi = MytRpc()

    if mytapi.init("192.168.1.100", 30202, 10) == True:
        try:
            # 导出节点树数据
            print("导出节点树数据")
            node_xml = mytapi.dumpNodeXml(True)  # True=导出所有节点
            if node_xml:
                # 保存节点树到文件
                with open("nodes.xml", "w", encoding='utf-8') as f:
                    f.write(node_xml)
                print("节点树数据导出成功")

            # 使用选择器查找节点
            print("使用选择器查找节点")
            selector = mytapi.create_selector()
            with selector:
                # 设置选择条件
                selector.addQuery_TextContainWith('登录')  # 文本包含"登录"
                selector.addQuery_Clickable(True)  # 可点击

                # 查找节点
                node = selector.execQueryOne(2000)  # 超时2000毫秒，返回第一个匹配节点
                if node is not None:
                    print("找到匹配节点")
                    print(f"节点信息: {node.getNodeJson()}")
                    # 点击节点
                    node.Click_events()
                    print("节点点击成功")
                else:
                    print("未找到匹配节点")

        finally:
            pass

if __name__ == "__main__":
    main()
```

按键码参考

```
# 常用按键码列表
KEYCODE_CALL = 5        # 拨号键
KEYCODE_ENDCALL = 6     # 挂机键
KEYCODE_HOME = 3        # Home键
KEYCODE_MENU = 82       # 菜单键
KEYCODE_BACK = 4        # 返回键
KEYCODE_SEARCH = 84     # 搜索键
KEYCODE_CAMERA = 27     # 拍照键
KEYCODE_POWER = 26      # 电源键
KEYCODE_VOLUME_UP = 24  # 音量增加键
KEYCODE_VOLUME_DOWN = 25# 音量减小键
KEYCODE_ENTER = 66      # 回车键
KEYCODE_DEL = 67        # 退格键
KEYCODE_TAB = 61        # Tab键
```

*Last updated on **Jan 27, 2026***